元胞自动机简介及其在数学模型中的应用

大学生数学建模基础课程

周吕文

极值学院

提要

1. 简介
   - 历史和应用
   - 引例
   - 定义
2. 实例
   - 森林火灾
   - 交通模拟
   - 应用展示
3. 总结
   - 补充
   - 总结
   - 练习

历史

- 最初的元胞自动机是由冯·诺依曼在 1950 年代为模拟生物细胞的自我复制而提出的. 但是并未受到学术界重视.
- 1970 年, 剑桥大学的约翰·何顿·康威设计了一个电脑游戏“生命游戏”后, 元胞自动机才吸引了科学家们的注意.
- 1983 年 S.Wolfram 发表了一系列论文. 对初等元胞机 256 种规则所产生的模型进行了深入研究, 并用熵来描述其演化行为, 将细胞自动机分为平稳型, 周期型, 混沌型和复杂型.

应用

- **社会学**: 元胞自动机经常用于研究个人行为的社会性, 流行现象. 例如人口迁移, 公共场所内人员的疏散, 流行病传播.
- **图形学**: 元胞自动机以其特有的结构的简单性, 内在的并行性以及复杂计算的能力成为密码学中研究的热点方向之一.
- **物理学**: 在物理学中, 元胞自动机已成功的应用于流体, 磁场, 电场, 热传导等的模拟. 例如格子气自动机.

数学建模竞赛中的应用

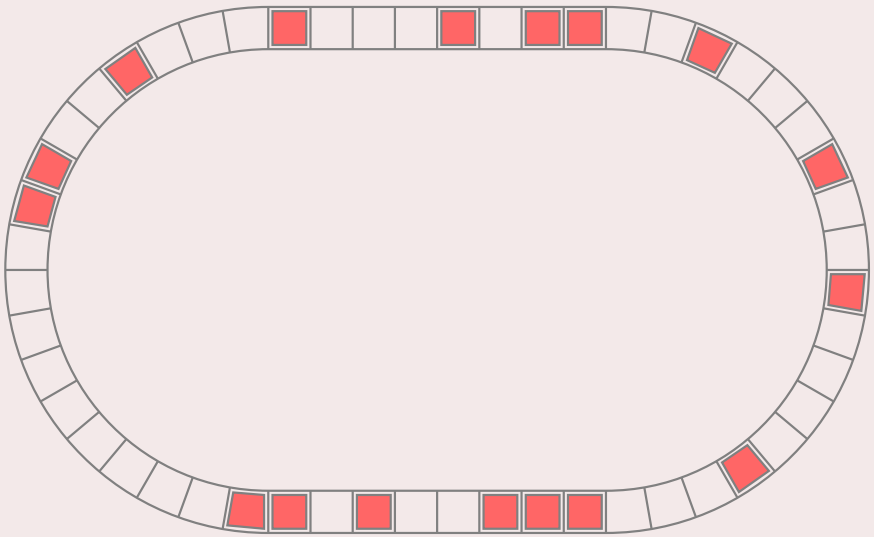
表: 近十来年 MCM 中用到元胞自动机的特等奖论文统计

年份题号	题目	特等奖论文数
2001MCM-B	逃避飓风	≥ 1
2003MCM-B	Gamma 刀最佳治疗方案	1
2005MCM-B	收费亭的最优数量	3
2007MCM-B	飞机座位方案	4
2009MCM-A	交通环岛的设计	≥ 2
2012MCM-B	沿着“大长河”露营	1
2014MCM-A	交通右行规则	6
2017MCM-C	协调和行驶	2

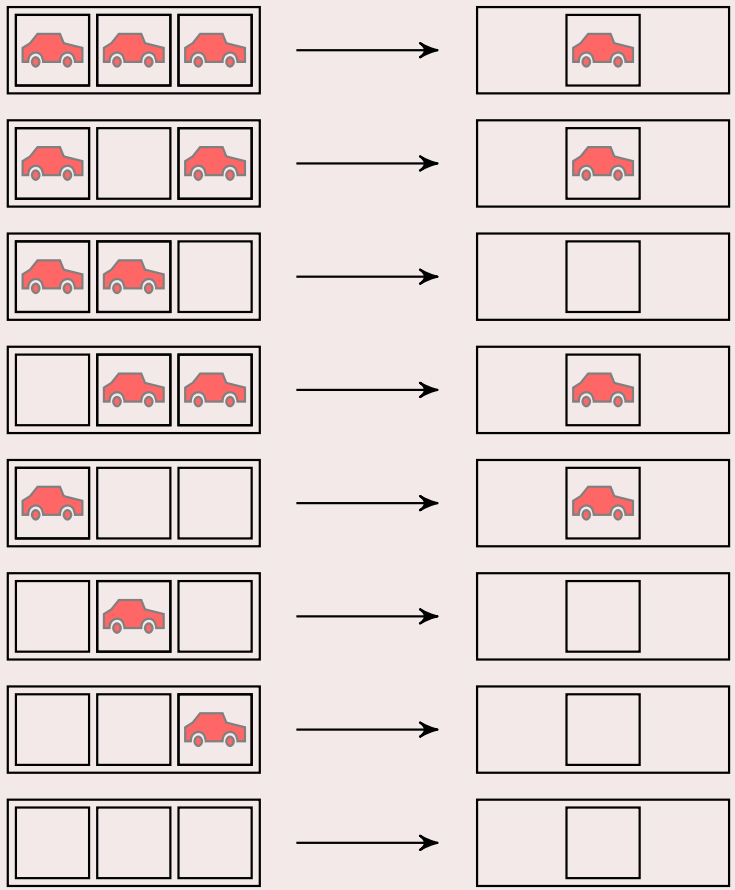
交通规则

定义

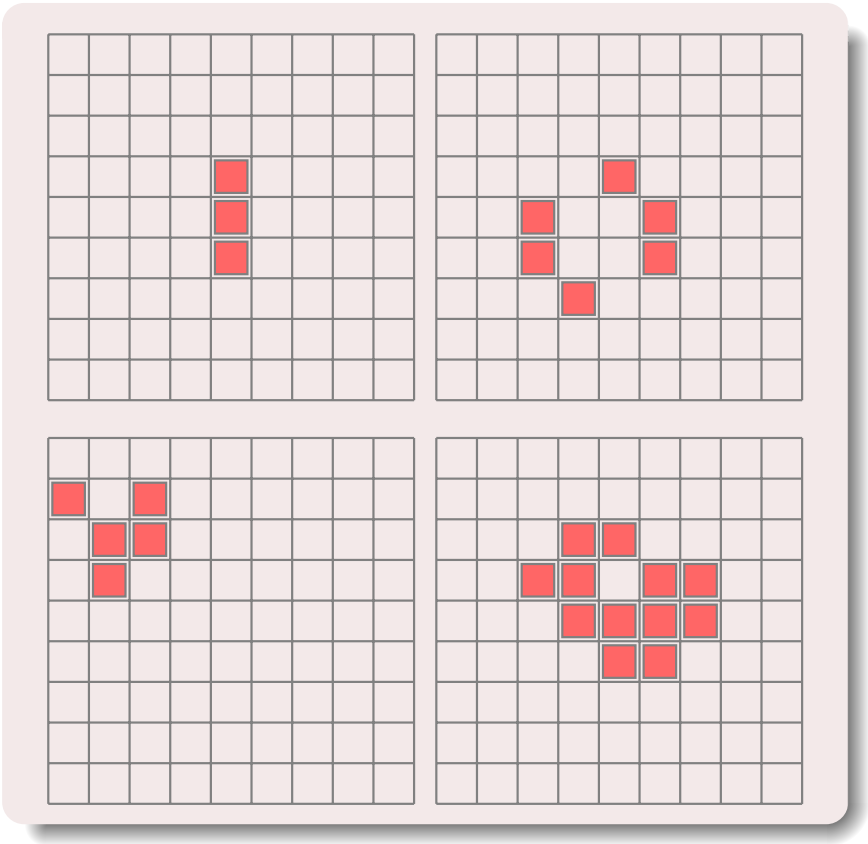
- 元胞分布于一维线性网格上.
- 元胞仅具有车和空两种状态.
- 元胞状态由周围两邻居决定.



规则



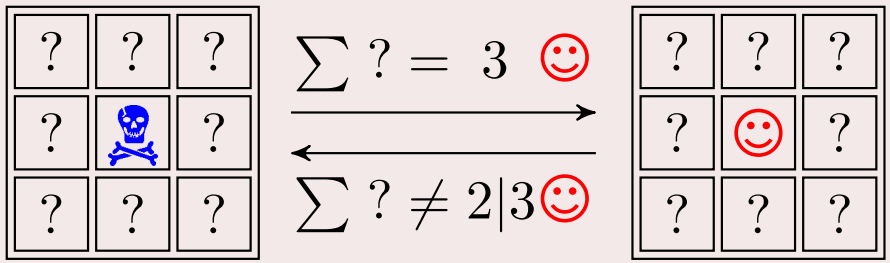
生命游戏



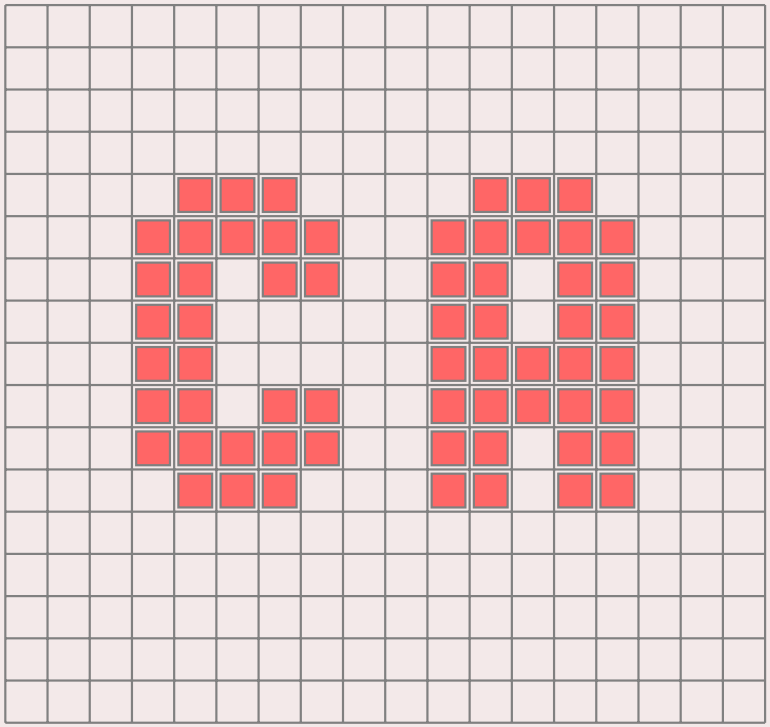
定义

- 元胞分布于二维方型网格上.
- 元胞仅具有生和死两种状态.
- 元胞状态由周围八邻居决定.

规则



什么是元胞自动机

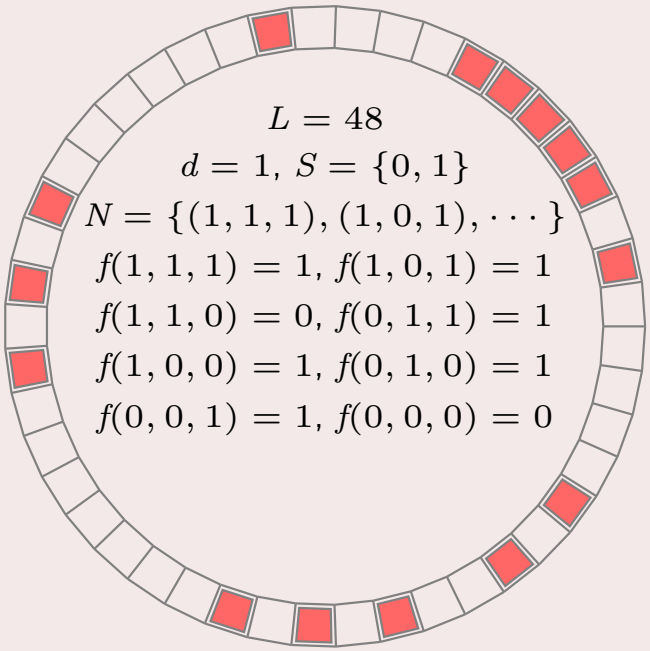


元胞自动机是离散的动力学系统

- 离散的系统: 元胞是定义在有限的时间和空间上的, 并且元胞的状态是有限.
- 动力学系统: 元胞自动机的举止行为具有动力学特征.
- 简单与复杂: 元胞自动机用简单规则控制相互作用的元胞模拟复杂世界.

构成要素

Rule 184

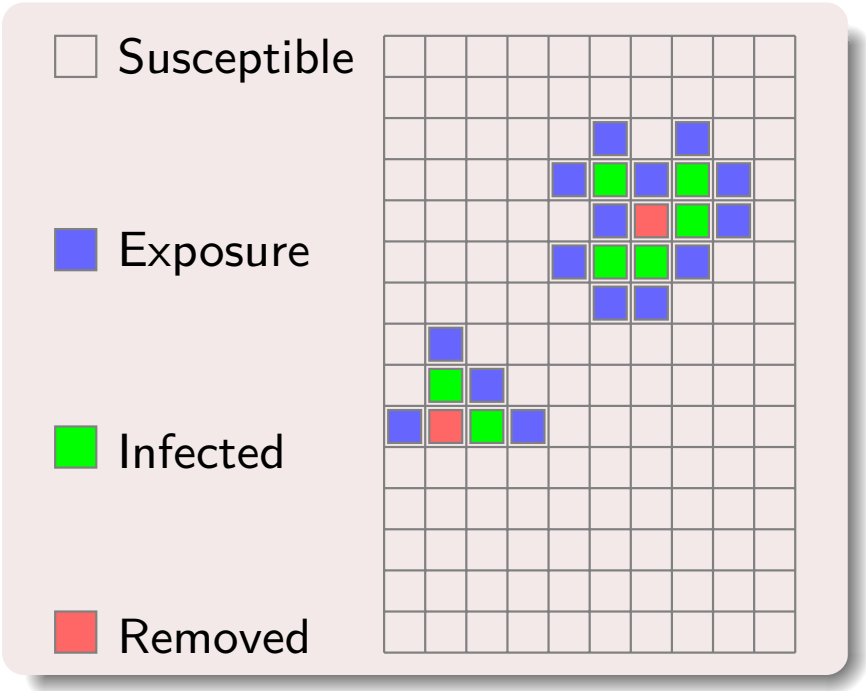


数学表示

$$A = (L,\ d,\ S,\ N,\ f)$$

- L: 元胞网格空间
- d: 元胞空间的维数
- S: 有限离散的状态集合
- N: 某邻域内所有元胞的集合
- f: 局部映射或局部规则

元胞 (Cell)

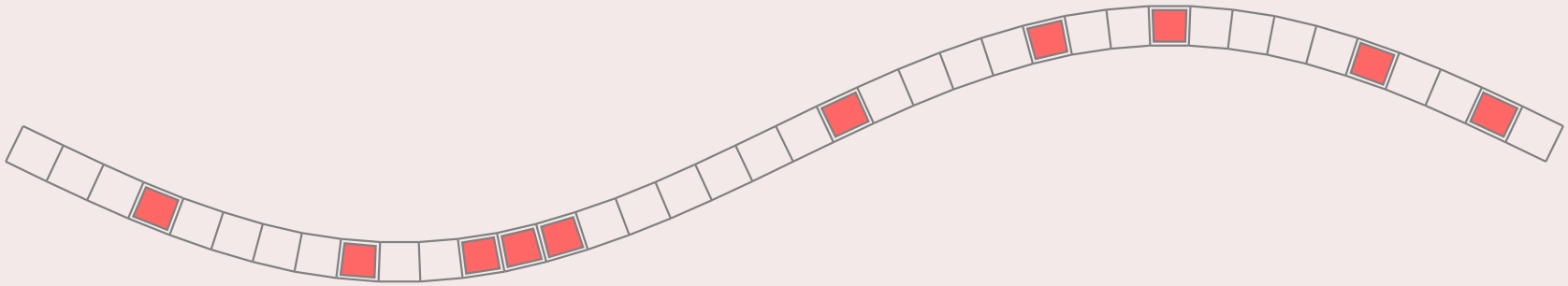


元胞是元胞自动机基本单元

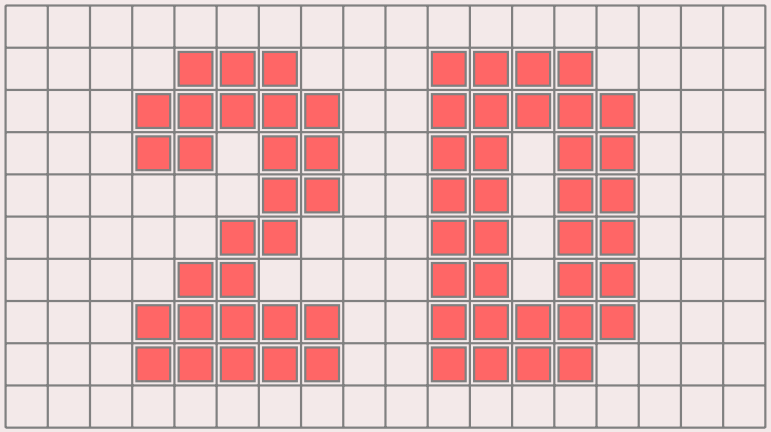
- **状态**: 每一个元胞都有记忆贮存状态的功能.
- **离散**: 简单情况下, 元胞只有两种可能状态; 较复杂情况下, 元胞具有多种状态.
- **更新**: 元胞的状态都安照动力规则不断更新.

网格 (Lattice)　不同维网格

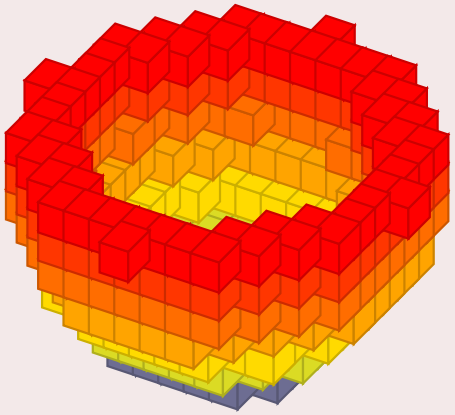
一维



二维

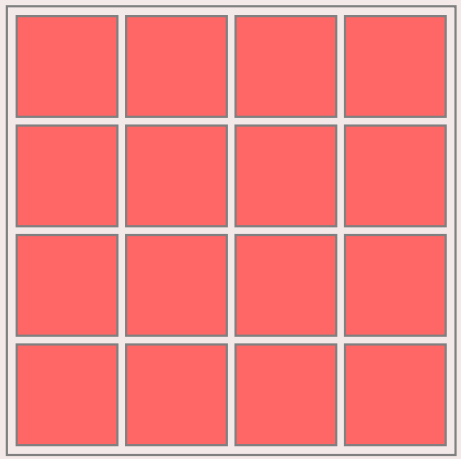


三维

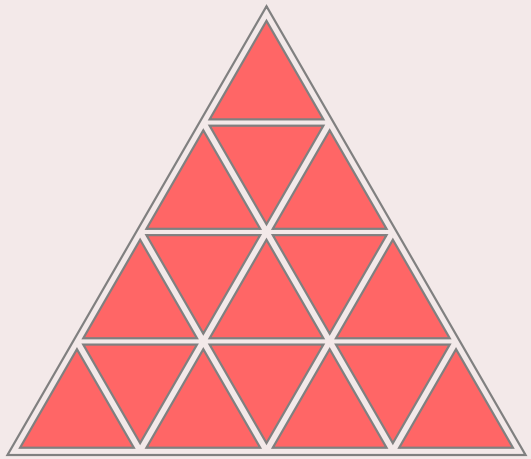


网格 (Lattice)　常用二维网格

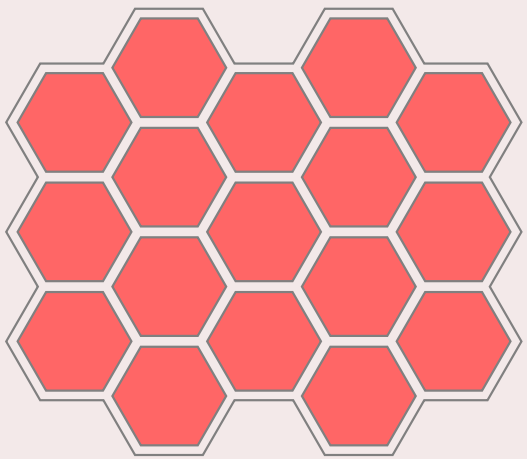
正方型网格



三角型网格

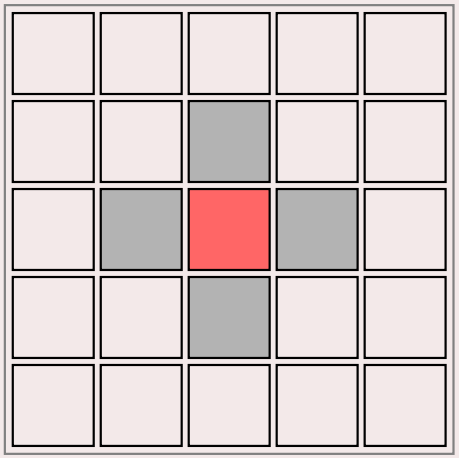


六边型网格

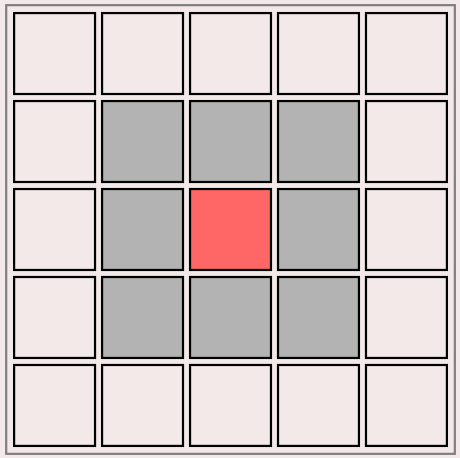


邻居 (Neighborhood)

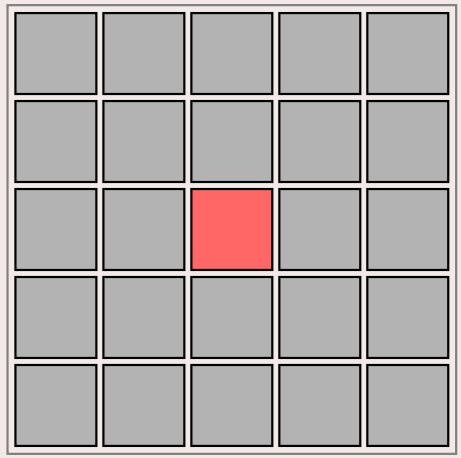
VonNeumann 邻居



Moore 邻居

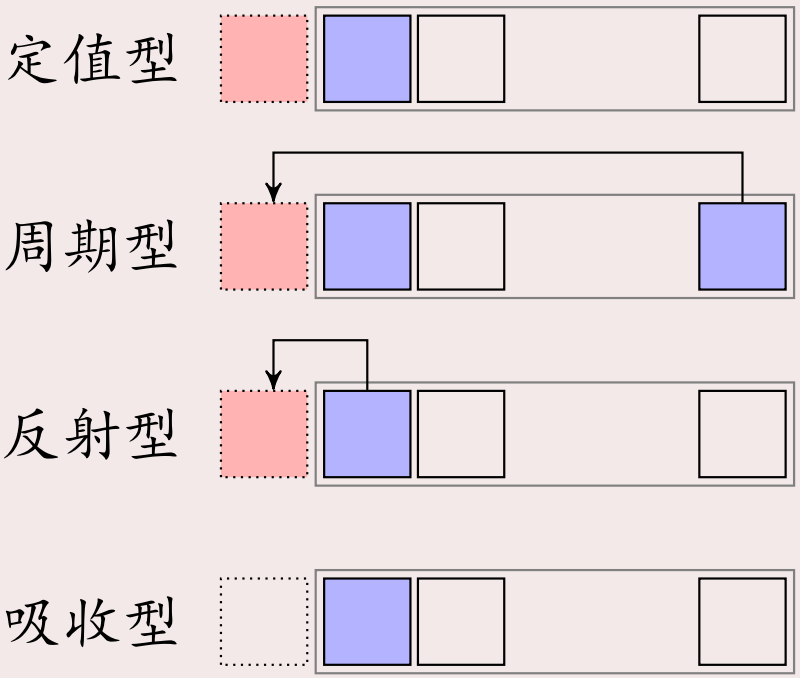


扩展 Moore 邻居



边界 (Boundary)

几种边界条件



二维 Moore 邻居周期边界条件

c_5		a_1	a_2	a_3	c_7	c_6
		a_8	a	a_4		
b_2	b_3	a_7	a_6	a_5		b_1
b	b_4					b_8
b_6	b_5					b_7
c_3					c_1	c_2
c_4					c_8	c

规则 (Rule)

定义

- 根据元胞当前状态及其邻居状况确定下一时刻该元胞状态的动力学函数, 简单讲, 就是一个状态转移函数.

分类

- **总和型**: 某元胞下时刻的状态取决于且仅取决于它所有邻居的当前状态以及自身的当前状态.
- **合法型**: 总和型规则属于合法型规则. 但如果把元胞自动机的规则限制为总和型, 会使元胞自动机具有局限性.

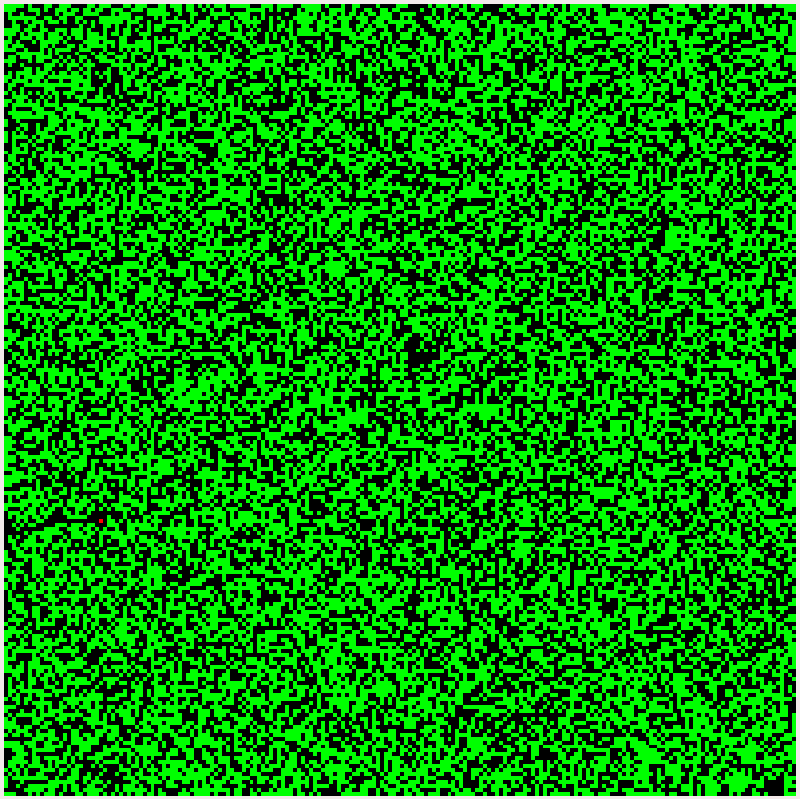
提要

1 简介
- 历史和应用
- 引例
- 定义

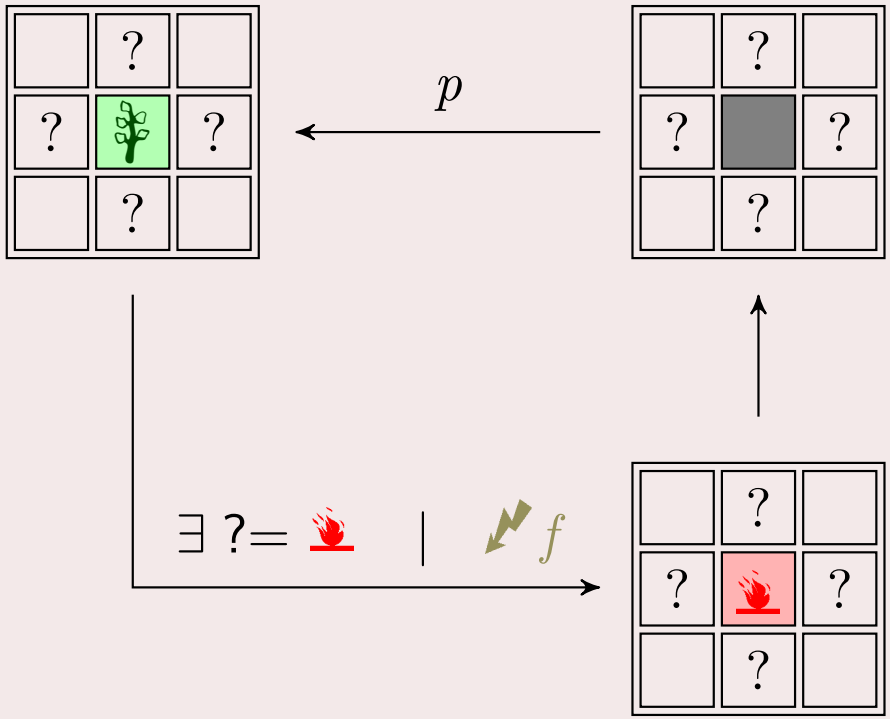
2 实例
- 森林火灾
- 交通模拟
- 应用展示

3 总结
- 补充
- 总结
- 练习

规则



规则



理论分析

三种状态元胞密度关系

$$\rho_f + \rho_e + \rho_t = 1$$

系统稳定条件

$$\rho_f = p\rho_e$$

树生长与火灾蔓延的时间尺度分离条件

$$f \ll p \ll T_{\mathrm{smax}}$$

程序实现

01	06	11	16	21
02	07	12	17	22
03	08	13	18	23
04	09	14	19	24
05	10	15	20	25

程序实现

05	10	15	20	25
01	06	11	16	21
02	07	12	17	22
03	08	13	18	23
04	09	14	19	24

01	06	11	16	21
02	07	12	17	22
03	08	13	18	23
04	09	14	19	24
05	10	15	20	25

上邻居

- `U = C([5, 1:4], :)`

程序实现

05	10	15	20	25
01	06	11	16	21
02	07	12	17	22
03	08	13	18	23
04	09	14	19	24

01	06	11	16	21
02	07	12	17	22
03	08	13	18	23
04	09	14	19	24
05	10	15	20	25

02	07	12	17	22
03	08	13	18	23
04	09	14	19	24
05	10	15	20	25
01	06	11	16	21

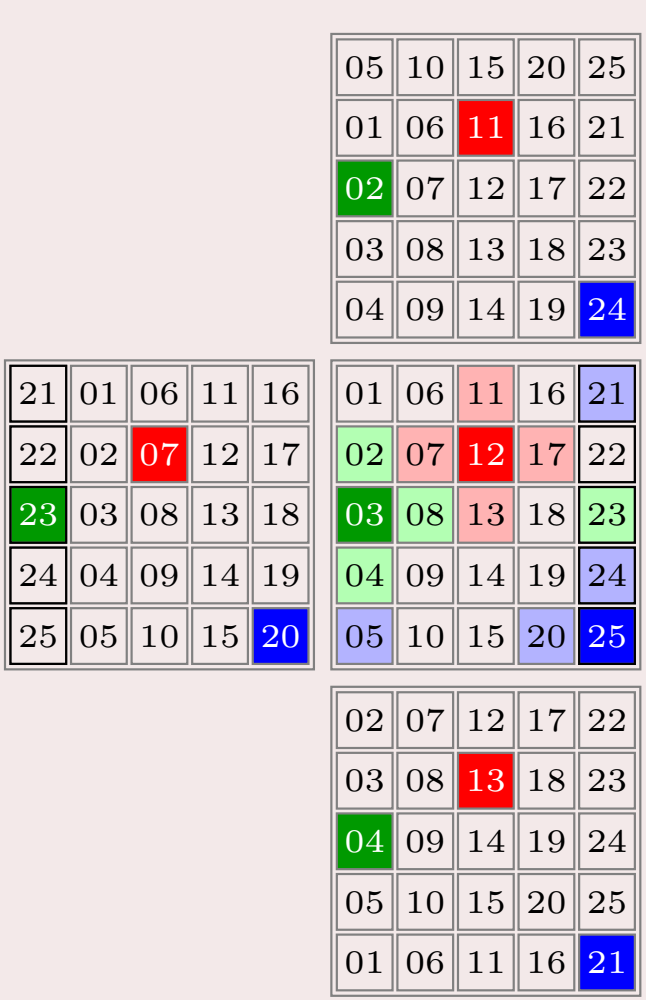
上邻居

- `U = C([5, 1:4], :)`

下邻居

- `D = C([2:5, 1], :)`

程序实现



上邻居

- `U = C([5, 1:4], :)`

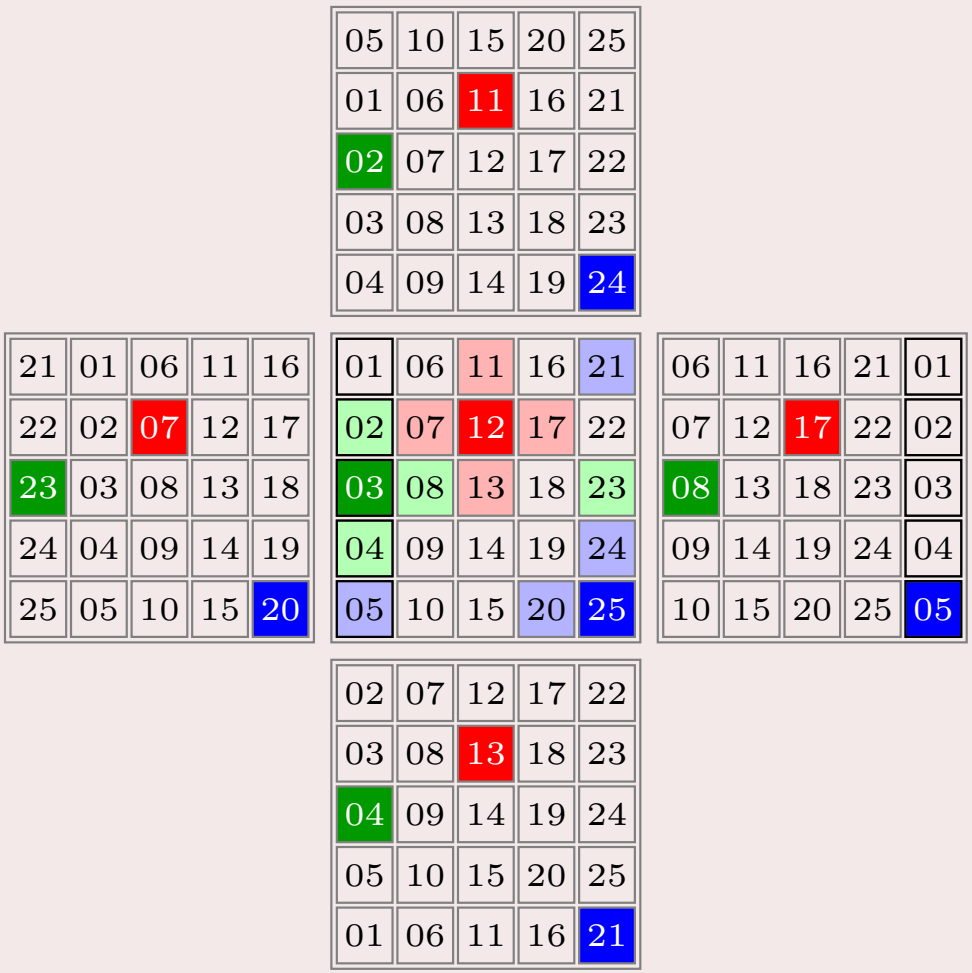
下邻居

- `D = C([2:5, 1], :)`

左邻居

- `L = C(:, [5, 1:4])`

程序实现



上邻居

- `U = C([5, 1:4], :)`

下邻居

- `D = C([2:5, 1], :)`

左邻居

- `L = C(:, [5, 1:4])`

右邻居

- `R = C(:, [2:5, 1])`

程序实现

```
01 n = 300;                                  % 定义表示森林的矩阵大小
02 Plight  = .000005; Pgrowth = .01;% 定义闪电和生长的概率
03 UL = [n 1:n-1];  DR = [2:n 1];    % 定义上左，下右邻居
04 veg=zeros(n,n);                           % 初始化表示森林的矩阵
05 imh = image(cat(3,veg,veg,veg)); % 可示化表示森林的矩阵
06 % veg = {empty=0 burning=1 green=2}
07 for i=1:3000                              % 主循环开始
08     sum =          (veg(UL,:)==1) + ...
09          (veg(:,UL)==1)   +   (veg(:,DR)==1) + ...
10                    (veg(DR,:)==1);% 求上下左右四个邻居和
11 % 根据规则更新森林矩阵: 树 = 树 - 着火的树 + 新生的树
12     veg = 2*(veg==2) - ...
13           ( (veg==2) & (sum>0|(rand(n,n)<Plight)) ) + ...
14           2*((veg==0) & rand(n,n)<Pgrowth) ;
15     set(imh, 'cdata', cat(3,(veg==1),(veg==2),zeros(n)) )
16     drawnow                               %可示化表示森林的矩阵
17 end                                       %主循环结束
```

程序实现

```
01 n = 300;                                  % 定义表示森林的矩阵大小
02 Plight  = .000005; Pgrowth = .01;% 定义闪电和生长的概率
03 UL = [n 1:n-1];  DR = [2:n 1];    % 定义上左，下右邻居
04 veg=zeros(n,n);                           % 初始化表示森林的矩阵
05 imh = image(cat(3,veg,veg,veg)); % 可示化表示森林的矩阵
06 % veg = {empty=0 burning=1 green=2}
07 for i=1:3000                              % 主循环开始
08     sum =          (veg(UL,:)==1) + ...
09          (veg(:,UL)==1)   +   (veg(:,DR)==1) + ...
10                     (veg(DR,:)==1);% 求上下左右四个邻居和
11 % 根据规则更新森林矩阵: 树 = 树 - 着火的树 + 新生的树
12     veg = 2*(veg==2) - ...
13           ( (veg==2) & (sum>0|(rand(n,n)<Plight)) ) + ...
14           2*((veg==0) & rand(n,n)<Pgrowth) ;
15     set(imh, 'cdata', cat(3,(veg==1),(veg==2),zeros(n)) )
16     drawnow                               %可示化表示森林的矩阵
17 end                                       %主循环结束
```

程序实现

```
01 n = 300;                                  % 定义表示森林的矩阵大小
02 Plight  = .000005; Pgrowth = .01;% 定义闪电和生长的概率
03 UL = [n 1:n-1];  DR = [2:n 1];    % 定义上左，下右邻居
04 veg=zeros(n,n);                           % 初始化表示森林的矩阵
05 imh = image(cat(3,veg,veg,veg)); % 可示化表示森林的矩阵
06 % veg = {empty=0 burning=1 green=2}
07 for i=1:3000                              % 主循环开始
08     sum =          (veg(UL,:)==1) + ...
09          (veg(:,UL)==1)   +   (veg(:,DR)==1) + ...
10                     (veg(DR,:)==1);% 求上下左右四个邻居和
11 % 根据规则更新森林矩阵: 树 = 树 - 着火的树 + 新生的树
12     veg = 2*(veg==2) - ...
13           ( (veg==2) & (sum>0|(rand(n,n)<Plight)) ) + ...
14           2*((veg==0) & rand(n,n)<Pgrowth) ;
15     set(imh, 'cdata', cat(3,(veg==1),(veg==2),zeros(n)) )
16     drawnow                               %可示化表示森林的矩阵
17 end                                       %主循环结束
```

程序实现

```
01 n = 300;                                  % 定义表示森林的矩阵大小
02 Plight  = .000005; Pgrowth = .01;% 定义闪电和生长的概率
03 UL = [n 1:n-1];  DR = [2:n 1];    % 定义上左，下右邻居
04 veg=zeros(n,n);                           % 初始化表示森林的矩阵
05 imh = image(cat(3,veg,veg,veg)); % 可示化表示森林的矩阵
06 % veg = {empty=0 burning=1 green=2}
07 for i=1:3000                              % 主循环开始
08     sum =          (veg(UL,:)==1) + ...
09          (veg(:,UL)==1)   +   (veg(:,DR)==1) + ...
10                    (veg(DR,:)==1);% 求上下左右四个邻居和
11 % 根据规则更新森林矩阵: 树 = 树 - 着火的树 + 新生的树
12     veg = 2*(veg==2) - ...
13           ( (veg==2) & (sum>0|(rand(n,n)<Plight)) ) + ...
14           2*((veg==0) & rand(n,n)<Pgrowth) ;
15     set(imh, 'cdata', cat(3,(veg==1),(veg==2),zeros(n)) )
16     drawnow                               %可示化表示森林的矩阵
17 end                                       %主循环结束
```

程序实现

```
01 n = 300;                                  % 定义表示森林的矩阵大小
02 Plight  = .000005; Pgrowth = .01;% 定义闪电和生长的概率
03 UL = [n 1:n-1];  DR = [2:n 1];    % 定义上左，下右邻居
04 veg=zeros(n,n);                           % 初始化表示森林的矩阵
05 imh = image(cat(3,veg,veg,veg)); % 可示化表示森林的矩阵
06 % veg = {empty=0 burning=1 green=2}
07 for i=1:3000                              % 主循环开始
08     sum =          (veg(UL,:)==1) + ...
09          (veg(:,UL)==1)   +   (veg(:,DR)==1) + ...
10                     (veg(DR,:)==1);% 求上下左右四个邻居和
11 % 根据规则更新森林矩阵: 树 = 树 - 着火的树 + 新生的树
12     veg = 2*(veg==2) - ...
13           ( (veg==2) & (sum>0|(rand(n,n)<Plight)) ) + ...
14           2*((veg==0) & rand(n,n)<Pgrowth) ;
15     set(imh, 'cdata', cat(3,(veg==1),(veg==2),zeros(n)) )
16     drawnow                               %可示化表示森林的矩阵
17 end                                       %主循环结束
```

程序实现

```
01 n = 300;                                  % 定义表示森林的矩阵大小
02 Plight  = .000005; Pgrowth = .01;% 定义闪电和生长的概率
03 UL = [n 1:n-1];  DR = [2:n 1];    % 定义上左，下右邻居
04 veg=zeros(n,n);                           % 初始化表示森林的矩阵
05 imh = image(cat(3,veg,veg,veg)); % 可示化表示森林的矩阵
06 % veg = {empty=0 burning=1 green=2}
07 for i=1:3000                              % 主循环开始
08     sum =          (veg(UL,:)==1) + ...
09          (veg(:,UL)==1)   +   (veg(:,DR)==1) + ...
10                    (veg(DR,:)==1);% 求上下左右四个邻居和
11 % 根据规则更新森林矩阵: 树 = 树 - 着火的树 + 新生的树
12     veg = 2*(veg==2) - ...
13           ( (veg==2) & (sum>0|(rand(n,n)<Plight)) ) + ...
14           2*((veg==0) & rand(n,n)<Pgrowth) ;
15     set(imh, 'cdata', cat(3,(veg==1),(veg==2),zeros(n)) )
16     drawnow                               %可示化表示森林的矩阵
17 end                                       %主循环结束
```

程序实现

```
01 n = 300;                                  % 定义表示森林的矩阵大小
02 Plight  = .000005; Pgrowth = .01;% 定义闪电和生长的概率
03 UL = [n 1:n-1];  DR = [2:n 1];    % 定义上左，下右邻居
04 veg=zeros(n,n);                           % 初始化表示森林的矩阵
05 imh = image(cat(3,veg,veg,veg)); % 可示化表示森林的矩阵
06 % veg = {empty=0 burning=1 green=2}
07 for i=1:3000                              % 主循环开始
08     sum =          (veg(UL,:)==1) + ...
09          (veg(:,UL)==1)   +   (veg(:,DR)==1) + ...
10                     (veg(DR,:)==1);% 求上下左右四个邻居和
11 % 根据规则更新森林矩阵: 树 = 树 - 着火的树 + 新生的树
12     veg = 2*(veg==2) - ...
13           ( (veg==2) & (sum>0|(rand(n,n)<Plight)) ) + ...
14           2*((veg==0) & rand(n,n)<Pgrowth) ;
15     set(imh, 'cdata', cat(3,(veg==1),(veg==2),zeros(n)) )
16     drawnow                               %可示化表示森林的矩阵
17 end                                       %主循环结束
```

程序实现

```
01 n = 300;                                  % 定义表示森林的矩阵大小
02 Plight  = .000005; Pgrowth = .01;% 定义闪电和生长的概率
03 UL = [n 1:n-1];  DR = [2:n 1];    % 定义上左，下右邻居
04 veg=zeros(n,n);                           % 初始化表示森林的矩阵
05 imh = image(cat(3,veg,veg,veg)); % 可示化表示森林的矩阵
06 % veg = {empty=0 burning=1 green=2}
07 for i=1:3000                              % 主循环开始
08     sum =          (veg(UL,:)==1) + ...
09          (veg(:,UL)==1)   +   (veg(:,DR)==1) + ...
10                    (veg(DR,:)==1);% 求上下左右四个邻居和
11 % 根据规则更新森林矩阵: 树 = 树 - 着火的树 + 新生的树
12     veg = 2*(veg==2) - ...
13           ( (veg==2) & (sum>0|(rand(n,n)<Plight)) ) + ...
14           2*((veg==0) & rand(n,n)<Pgrowth) ;
15     set(imh, 'cdata', cat(3,(veg==1),(veg==2),zeros(n)) )
16     drawnow                               %可示化表示森林的矩阵
17 end                                       %主循环结束
```

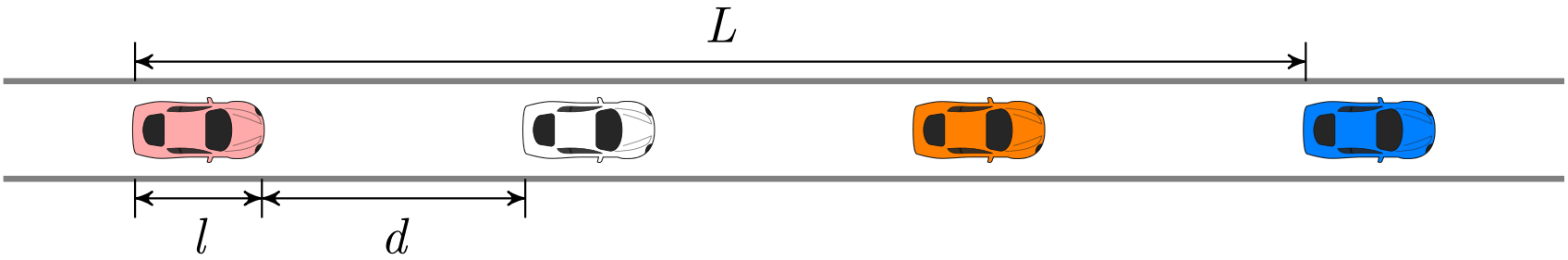
程序实现

```
01 n = 300;                                  % 定义表示森林的矩阵大小
02 Plight  = .000005; Pgrowth = .01;% 定义闪电和生长的概率
03 UL = [n 1:n-1];  DR = [2:n 1];    % 定义上左，下右邻居
04 veg=zeros(n,n);                           % 初始化表示森林的矩阵
05 imh = image(cat(3,veg,veg,veg)); % 可示化表示森林的矩阵
06 % veg = {empty=0 burning=1 green=2}
07 for i=1:3000                              % 主循环开始
08     sum =          (veg(UL,:)==1) + ...
09          (veg(:,UL)==1)   +   (veg(:,DR)==1) + ...
10                     (veg(DR,:)==1);% 求上下左右四个邻居和
11 % 根据规则更新森林矩阵: 树 = 树 - 着火的树 + 新生的树
12     veg = 2*(veg==2) - ...
13           ( (veg==2) & (sum>0|(rand(n,n)<Plight)) ) + ...
14           2*((veg==0) & rand(n,n)<Pgrowth) ;
15     set(imh, 'cdata', cat(3,(veg==1),(veg==2),zeros(n)) )
16     drawnow                               %可示化表示森林的矩阵
17 end                                       %主循环结束
```

程序实现

```
01 n = 300;                                  % 定义表示森林的矩阵大小
02 Plight  = .000005; Pgrowth = .01;% 定义闪电和生长的概率
03 UL = [n 1:n-1];  DR = [2:n 1];    % 定义上左，下右邻居
04 veg=zeros(n,n);                           % 初始化表示森林的矩阵
05 imh = image(cat(3,veg,veg,veg)); % 可示化表示森林的矩阵
06 % veg = {empty=0 burning=1 green=2}
07 for i=1:3000                              % 主循环开始
08     sum =          (veg(UL,:)==1) + ...
09          (veg(:,UL)==1)   +   (veg(:,DR)==1) + ...
10                     (veg(DR,:)==1);% 求上下左右四个邻居和
11 % 根据规则更新森林矩阵: 树 = 树 - 着火的树 + 新生的树
12     veg = 2*(veg==2) - ...
13           ( (veg==2) & (sum>0|(rand(n,n)<Plight)) ) + ...
14           2*((veg==0) & rand(n,n)<Pgrowth) ;
15     set(imh, 'cdata', cat(3,(veg==1),(veg==2),zeros(n)) )
16     drawnow                               %可示化表示森林的矩阵
17 end                                       %主循环结束
```

交通概念: 车距和密度



车距: 相临两车, 后车头到前车尾的距离

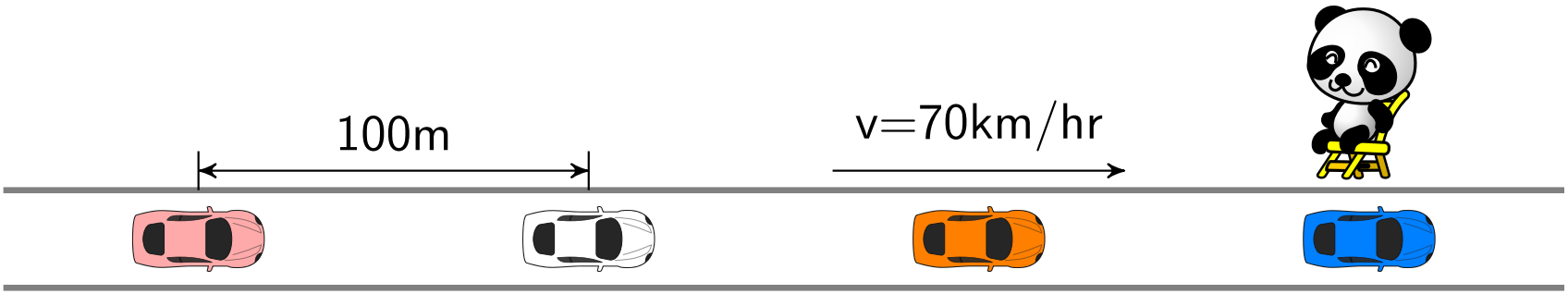
$$d = \frac{L - Nl}{N} = \frac{1}{\rho} - l$$

密度: 单位长度上分布的车辆数

$$\rho = \frac{N}{L} = \frac{1}{d + l}, \qquad \rho_{\max} = \frac{1}{l}$$

交通概念: 流量方程

如果周老师坐在一条公路的边上, 公路上的车速为 70km/hr, 车流密度为 10 veh/km. 每小时能从周老师身边驶过多少辆车?



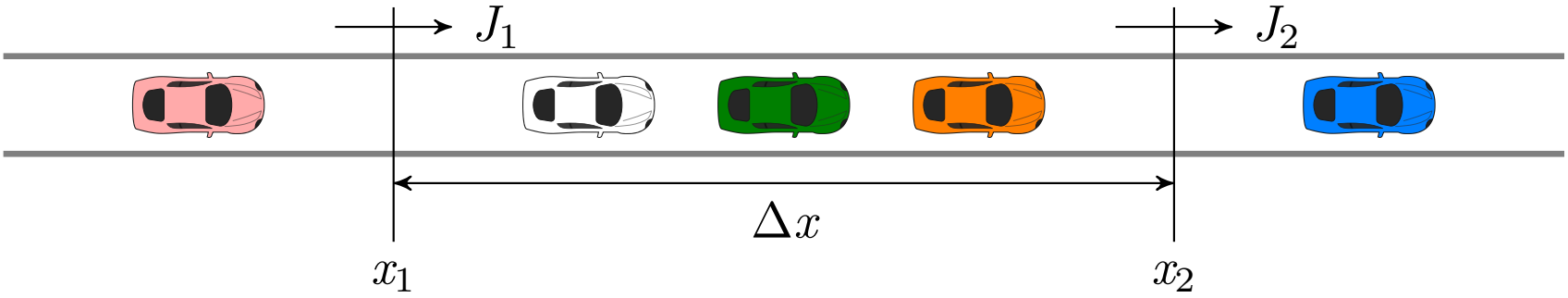
流量方程: 单位时间内通过某路段的车辆数

$$J = \rho v$$

- 量纲: [veh/hr] = [veh/km][km/h]
- $\rho = 0 \Longrightarrow J = 0$, $\rho = \rho_{\max} \Longrightarrow v = 0 \Longrightarrow J = 0$

交通概念: 守恒方程

长为 $\Delta x = x_2 - x_1$ 的一段公路, 在时间间隔 $\Delta t = t_2 - t_1$ 里, 车辆数 N 变化如何?



$$(J_{x_1} - J_{x_2})\Delta t = \Delta N = (\rho_{t_2} - \rho_{t_1})\Delta x \Longrightarrow \frac{J_{x_1} - J_{x_2}}{\Delta x} = \frac{\rho_{t_2} - \rho_{t_1}}{\Delta t}$$

守恒方程

$$J_x + \rho_t = (\rho v)_x + \rho_t = 0$$

交通概念: 宏观连续模型

Lighthill-Whitham-Richards 模型

$$\frac{\partial \rho(x,t)}{\partial t} + \frac{\partial q(x,t)}{\partial x} = 0$$

$$J = q(\rho, v) = q\big(\rho, v(\rho)\big) = q(\rho)$$

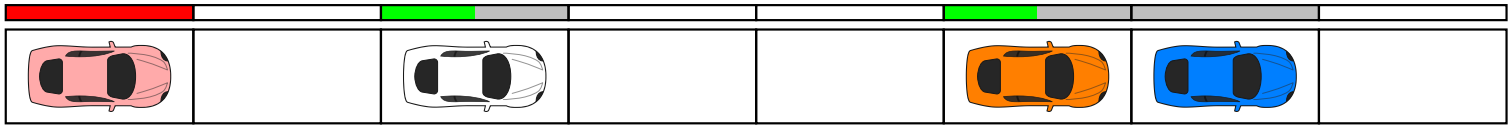
Greenshields 速度函数

$$v(\rho) = v_{\max}\left(1 - \frac{\rho}{\rho_{\max}}\right)$$

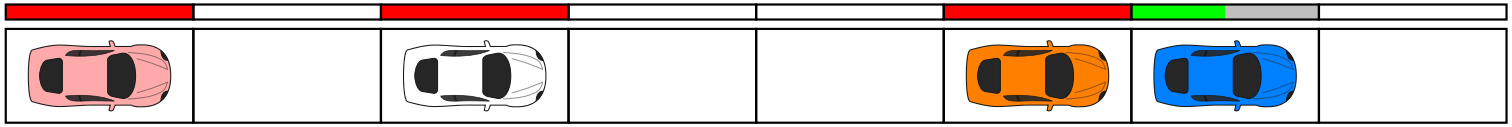
$$J(\rho) = v_{\max}\left(\rho - \frac{\rho^2}{\rho_{\max}}\right)$$

规则

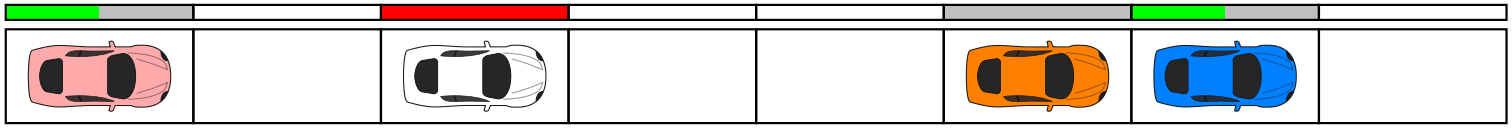
0. 初始状态: $v_1 = 2,\ v_2 = 1,\ v_3 = 1,\ v_4 = 0$



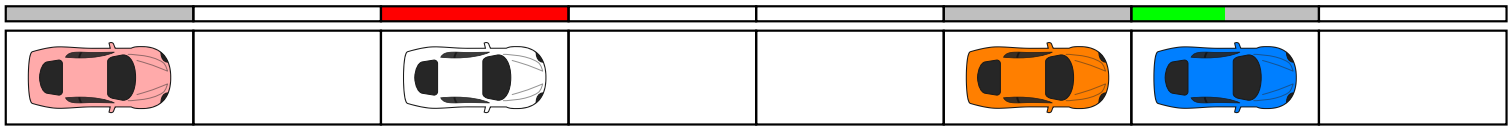
1. 加速规则: $v_n = \min\{v_{\max}, v_n + 1\}, \quad v_{\max} = 2$



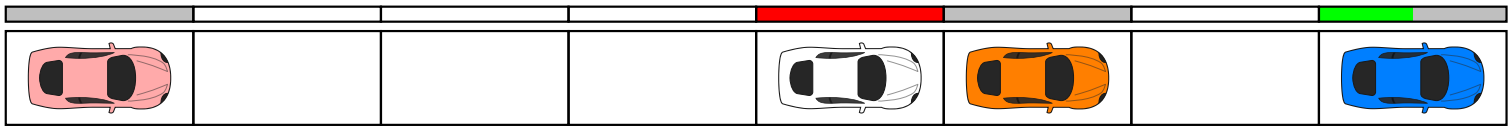
2. 防止碰撞: $v_n = \min\{v_n, d_n - 1\}$



3. 随机减速: $v_n \overset{p}{=} \max\{v_n - 1, 0\}, \qquad \mathsf{rand}_1 \le p$



4. 位置更新: $x_n = x_n + v_n$



理论分析

由车道长，车数，最大速度
$L = 36,\ N = 12,\ v_{\max} = 5,$
容易求得平均密度和间距
$\rho = \frac{N}{L} = \frac{1}{3},\ d = \frac{L-N}{N} = 2$
如何求平均速度和车流量?

临界密度

$$\rho_c = \frac{1}{v_{\max}+1} = \frac{1}{5+1}$$

平均速度

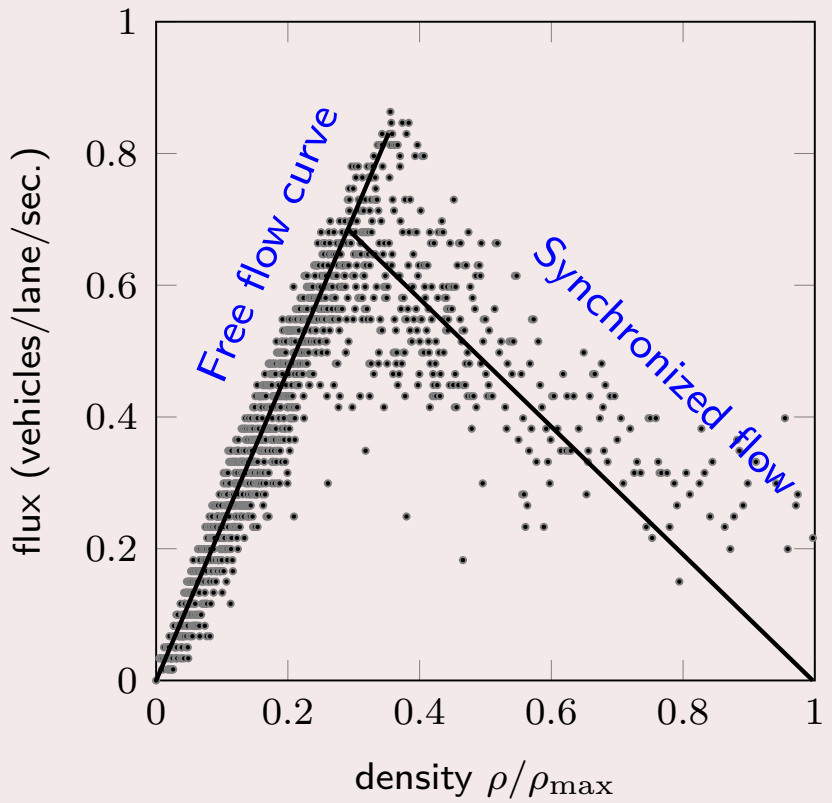
$$v = \min\{v_{\max}, d\} = 2$$

车流量

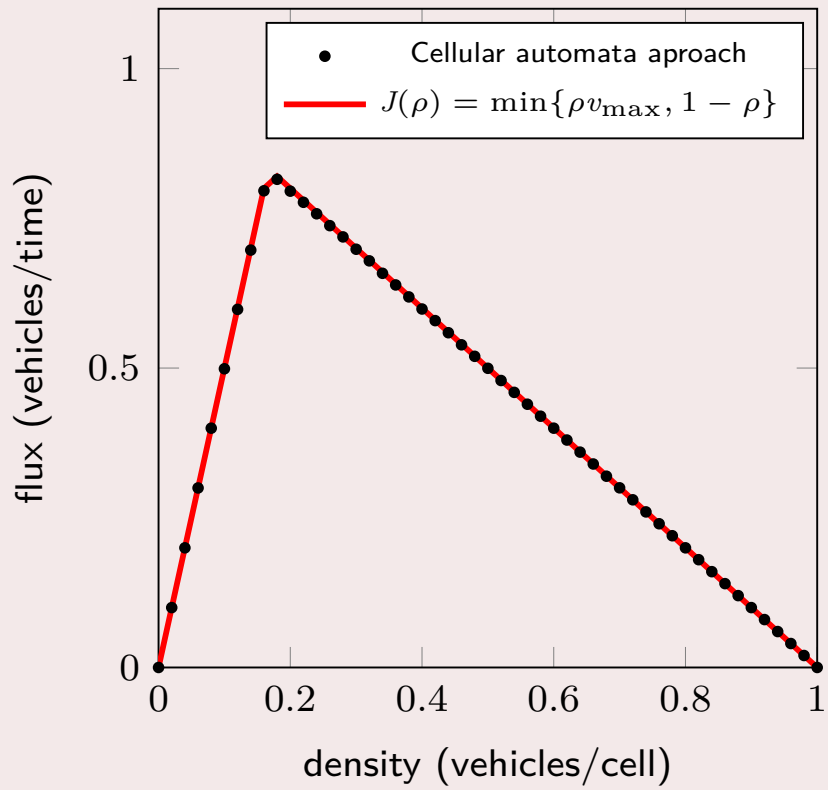
$$J = \min\{\rho v_{\max}, \rho d\} = 2/3$$
$$= \min\{\rho v_{\max}, 1-\rho\}$$

结果分析: 密度与流量

观测数据

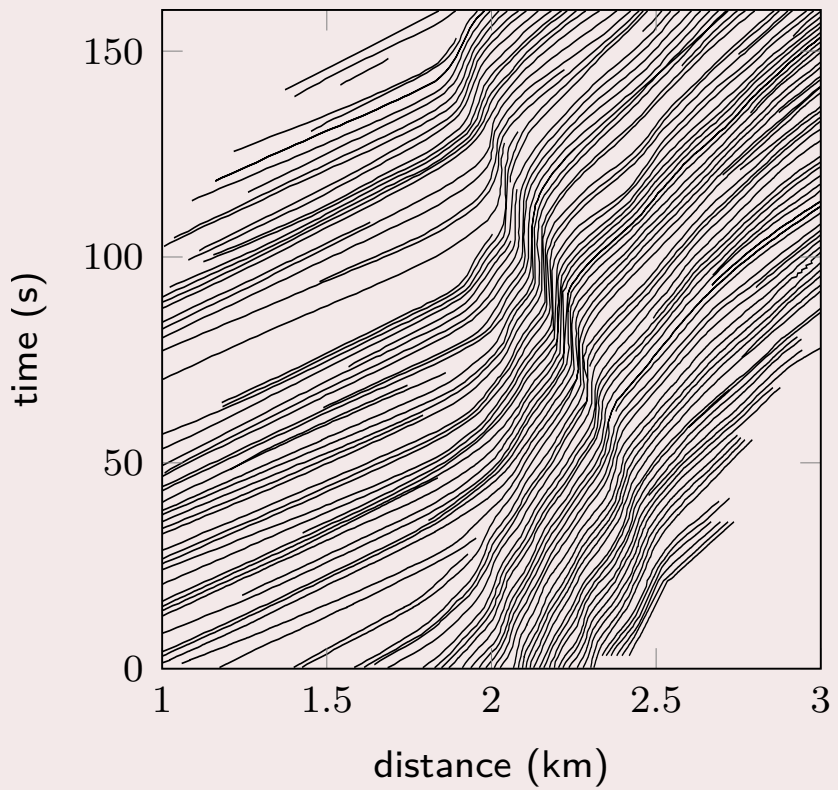


理论值及 CA 预测值

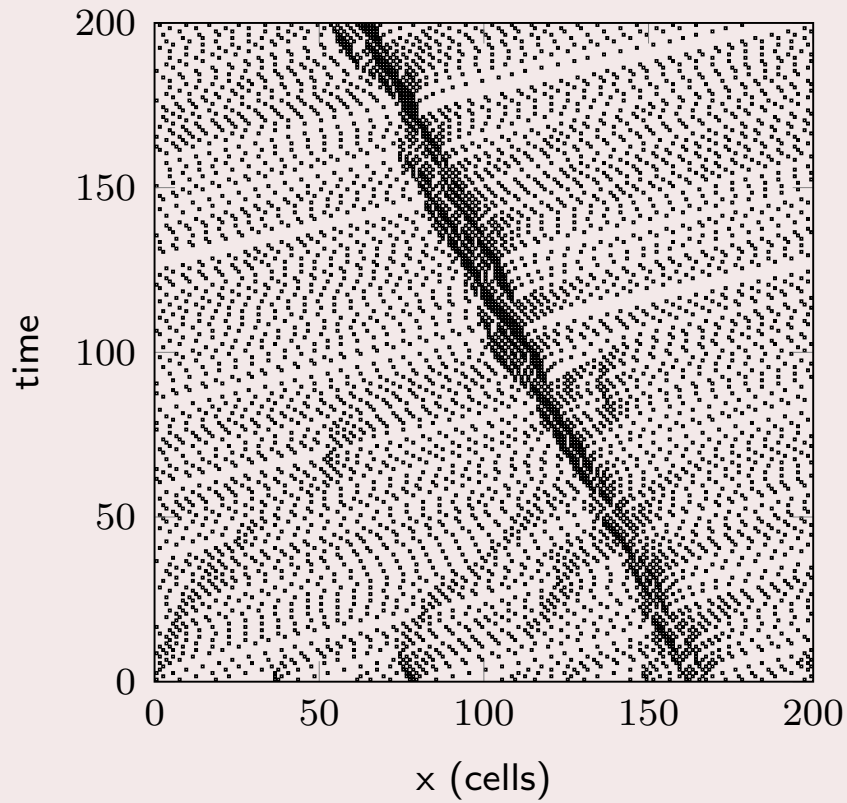


结果分析: 时空轨迹

航拍数据



CA 计算结果 ($p_{\text{brake}} = 0.15$)



程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

程序实现

ns.m

```
01 function flux = ns(rho,p,L,tmax)% rho=0.2;p=0.2;L=72;tmax=72
02 ncar = round(L*rho);             % ncar=L*rho
03 x = sort(randperm(L, ncar));     % 1:ncar中随机不重复的L个数
04 vmax = 5;                        % 最大速度
05 v = vmax * ones(1, ncar);        % 初始化所有车速度为vmax
06 for t = 1:tmax
07     v = min(v+1, vmax);              % 加速规则
08     gaps = gaplength(x,L,ncar);
09     v = min(v, gaps-1);              % 防止碰撞
10     v = max(v- (rand(1,ncar)<p), 0);% 随机减速 binornd
11     x = x + v;                       % 位置更新
12     x(x>L) = x(x>L) - L;             % 周期边界
13     flux = flux + sum(v)/L;          % 空间平均
14 end
15 flux = flux / tmax;              % 时间平均
```

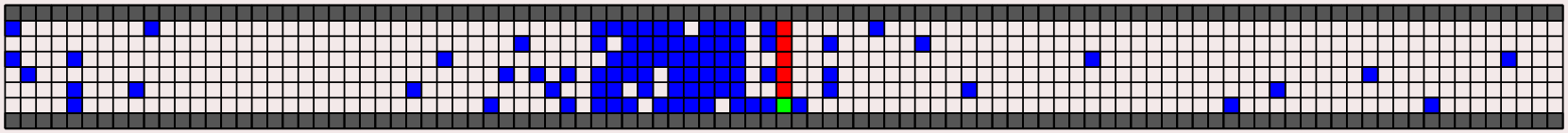
```
16 function gaps = gaplength(x, L, ncar)
17 gaps = zeros(1, ncar);
18 gaps = x([2:end 1]) -x;          % d(i) = x(i+1)-x(i)
19 gaps(gaps<=0) = gaps(gaps<=0)+L;% d(i) = d(i) + L, if d(i)<0
```

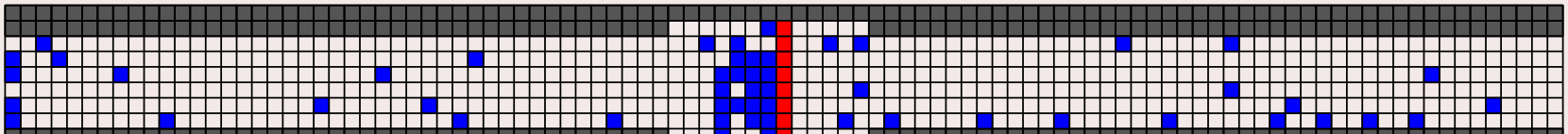
交通问题

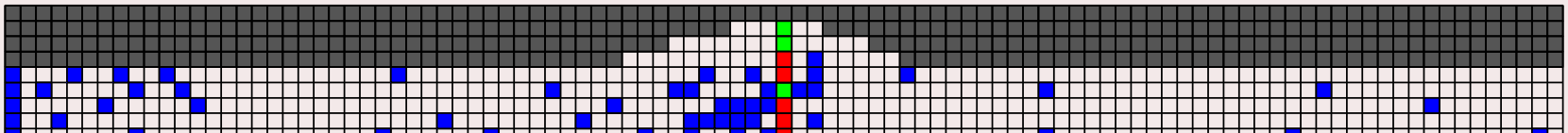
MCM2005B

MCM2009A

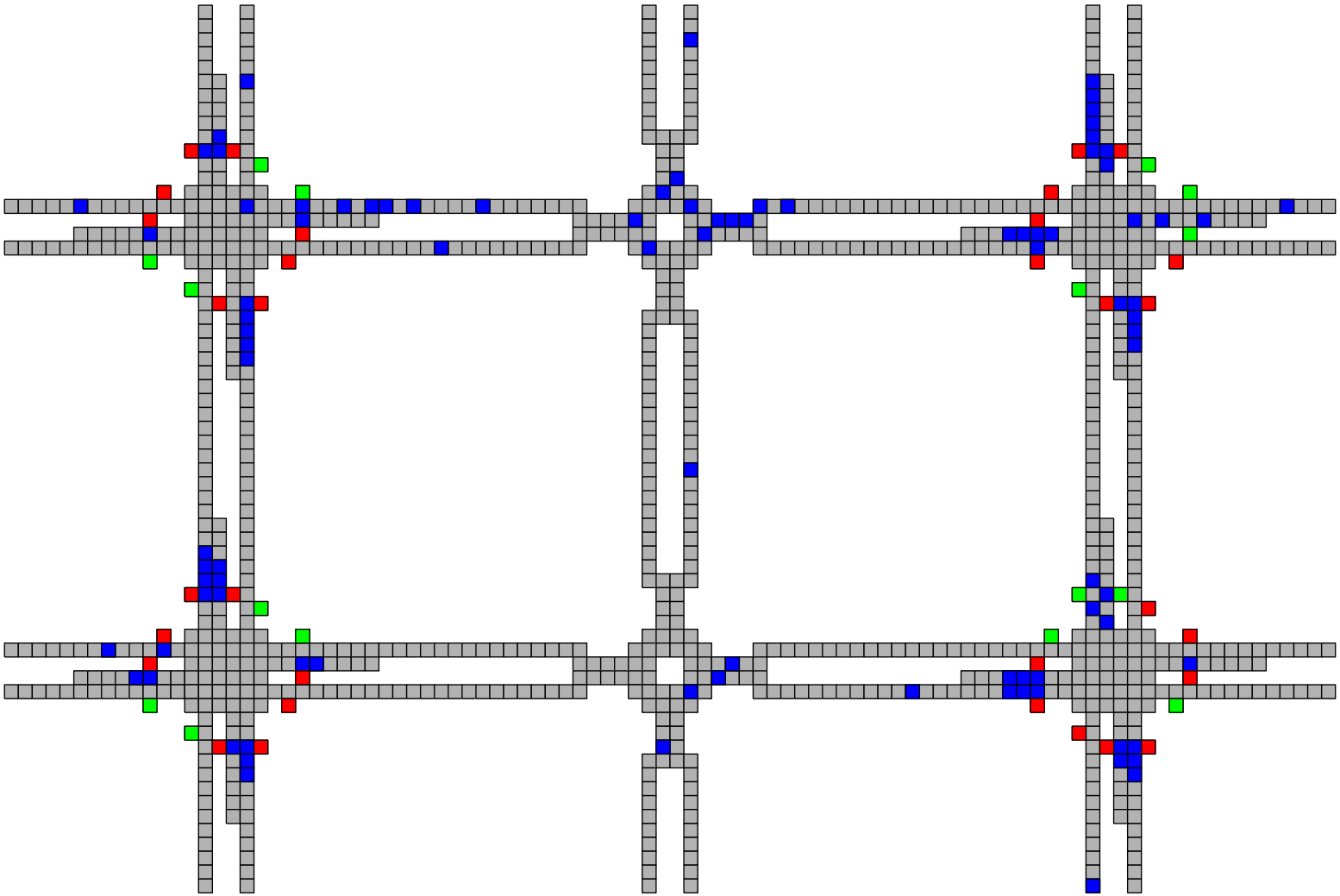
交通问题: 收费亭最优数量





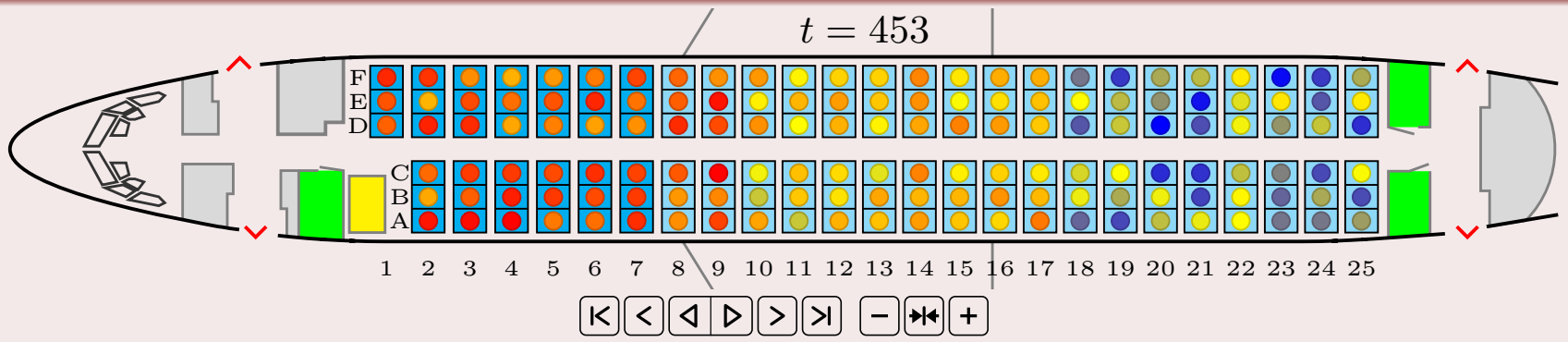


交通问题: 城市的交通网络

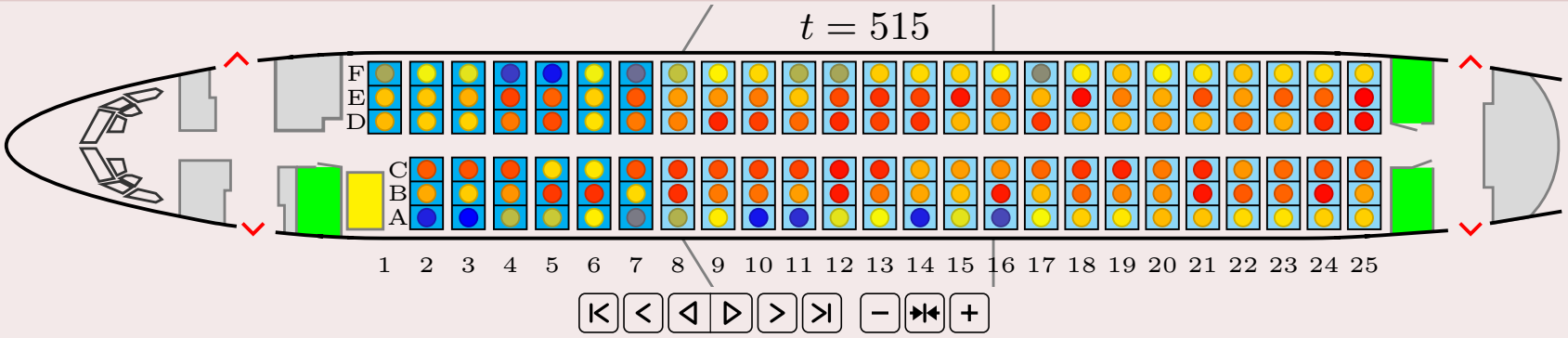


登机问题: boeing 737-400

Back to Front



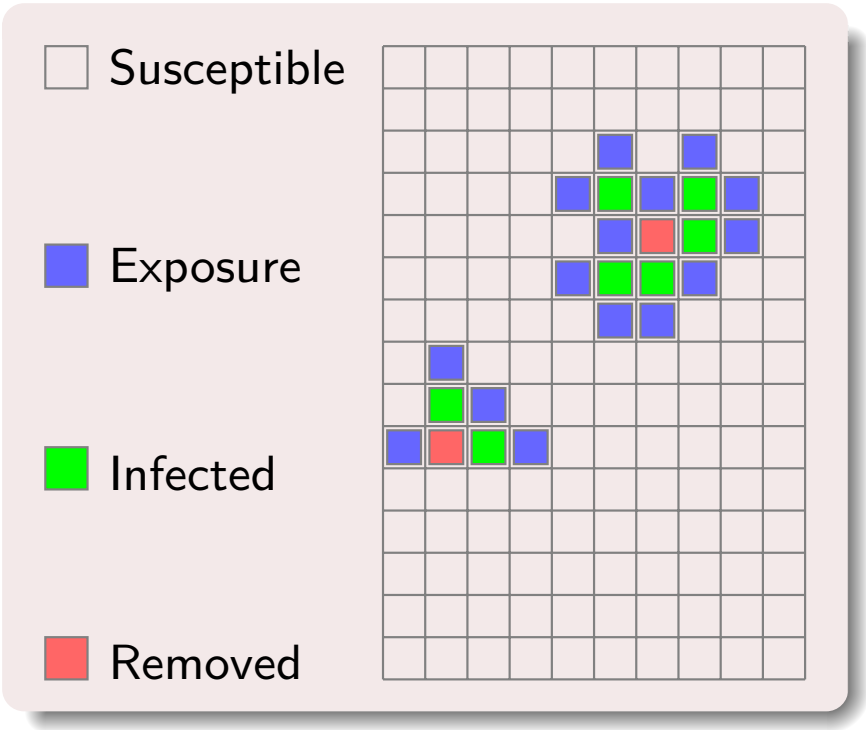
Window First



提要

1. 简介
   - 历史和应用
   - 引例
   - 定义
2. 实例
   - 森林火灾
   - 交通模拟
   - 应用展示
3. 总结
   - 补充
   - 总结
   - 练习

传染病元胞自动机模型

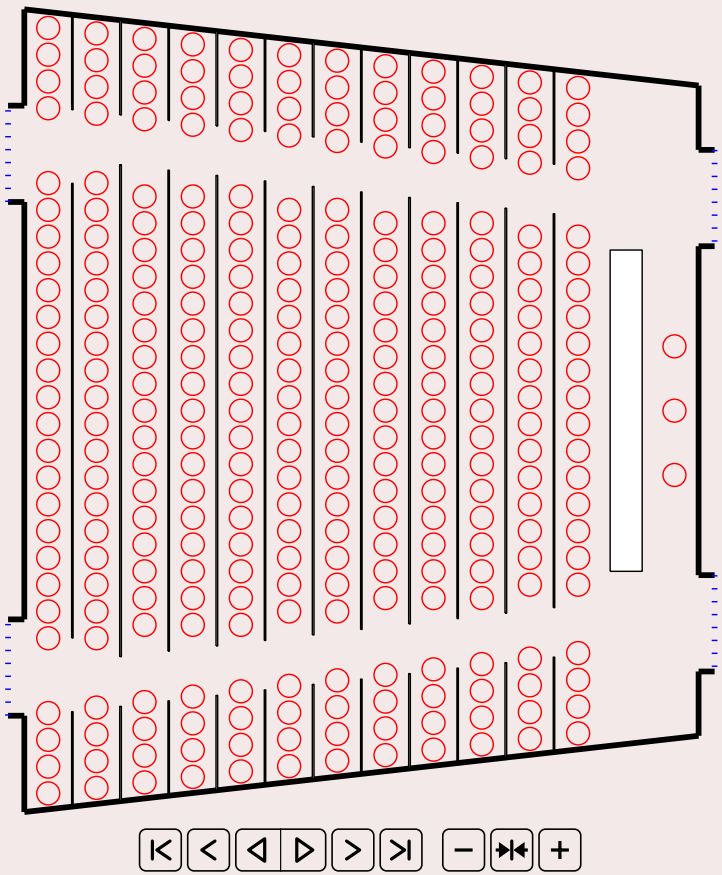


规则

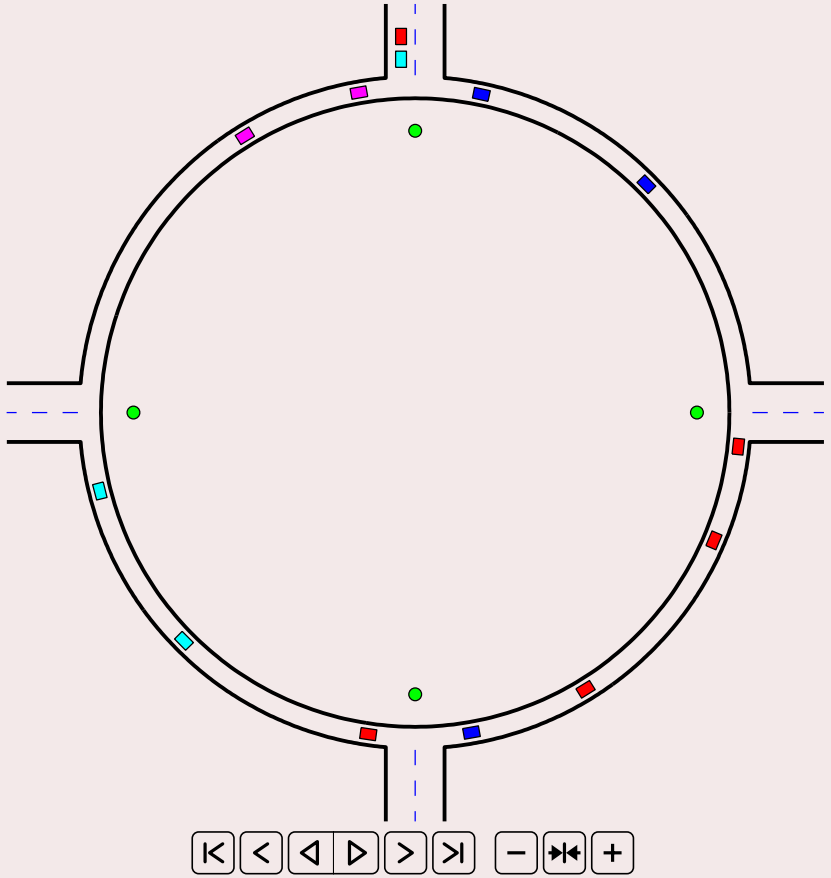
- 二维网络 VonNeumann 邻居.
- 四种状态: 易感 (S), 潜伏 E, 感染 I, 移除 R.
- 邻居中有感染 I, 则 S 以 p 的概率 $S \to E$.
- 经过确定的时间步 $E \to I$.
- 经过确定的时间步 $I \to R$.

与粒子方法的差别

疏散问题



交通问题



总结

特点

- 离散的空间, 离散的时间.
- 离散有限的状态.
- 同质的元胞.
- 局部的作用, 同步的计算.

注意

- 元胞自动机比较适合解决具有空间离散特点的动力学问题.
- 根据问题适当改造元胞自动机, 可使应用范围更广.
- 不要在不适当的问题上迁强地使用元胞自动机.

练习

简单

1. 建立 SIR 传染病元胞自动机模型并模拟, 分析参数和结果.
2. 制定更换车道规则, 实现双车道圆环内的 NS 交通流模拟.

较难

1. 美赛 2015A 根除埃博拉问题.
2. 国赛 2013A 车道被占用问题.

挑战

1. 自学 LBM, 并解决国赛 2011A 城市表层土壤重金属污染.
2. 第三届全国计算机仿真大赛: 口取消左转能否缓解拥堵.

Thank You!!!